Inventory Routing

March 2024

1 Abstract

The paper presents a multi-start iterated local search for solving the inventory routing problem with logistic ratio. The proposed optimisation algorithm utilise serval components include an application of the Lagrangian relaxation where violation on the stockout constratins is permitted, a devliery quantity reoptimisation procedure, and serval local search operators integrates both routing decision and quantity decision. The computational experiements were conducted on 978 small instances with 5 to 50 customer and 150 large instances with 50 to 200 customers. The results shown the proposed algorithm outperformance all instances on previous available metaheuristics for large instances and produces competitive results on small instances. We also give our attempt on integrating the hybrid genetic search from capacited vehicle routing problem into the proposed multi-started iterated local search. With hybrid genetic search, the solution quality can still be improved by certain percentage.

2 Introduction

The inventory routing problem (IRP) integrates inventory replenishment, delivery scheduling, and vehicle routing over a finite planning horizon. The supplier decides when to serve each customer, how much product to deliver, and how to construct the delivery routes. These decisions are interdependent: delivery quantities affect customer inventories and vehicle capacity, while visit timing and customer grouping influence routing cost and future replenishment requirements (Bertazzi and Speranza, 2012; Coelho et al., 2014).

Most classical IRP formulations minimize an absolute cost measure, commonly the sum of routing and inventory-holding costs (Archetti et al., 2012; Archetti and Speranza, 2016). In some applications, however, inventory costs are difficult to estimate or are small relative to transportation costs. In such cases, an alternative performance measure is the *logistic ratio* (LR), defined as the distribution cost per unit of product delivered. The LR was used in the ROADEF/EURO 2016 Air Liquide challenge, a rich industrial distribution problem. The single-supplier IRP with logistic ratio studied in this paper follows the academic formulation introduced by Archetti et al. (2017). Although both

settings use a cost-per-unit-delivered objective, they differ in their operational assumptions and constraints.

For a feasible distribution plan s, let $C(s)$ denote its total routing cost and $Q(s)$ the total quantity delivered over the planning horizon. The logistic ratio is

$$LR(s) = \frac{C(s)}{Q(s)}, \qquad Q(s) > 0. \tag{1}$$

Because delivery quantities are decision variables, the denominator is endogenous. Minimizing the LR therefore evaluates routing cost relative to the total quantity delivered and can result in replenishment decisions different from those obtained by minimizing routing cost alone. Archetti et al. (2017) also motivate the LR as a way to mitigate a finite-horizon effect of conventional IRP objectives: under total-cost minimization, additional deliveries near the end of the planning horizon may be unattractive when the associated consumption occurs beyond the horizon.

The purpose of this paper is to develop and evaluate a multi-start iterated local search (MSILS) for the single-supplier IRP-LR. The existing literature includes both exact methods and heuristic methods for the problem. In particular, Alvarez et al. (2018) already proposed iterated local search and simulated annealing for the LR objective. Accordingly, the contribution of this paper is not the first heuristic for the IRP-LR. It is the design and integration of the construction, search, penalty, and rebalancing procedures described in this paper, and their computational evaluation against the existing heuristic baseline.

Organisation of the paper. [Insert a brief paragraph here describing the remaining sections of the paper once the final section order and labels have been confirmed.]

3 Problem Definition

We study the single-product IRP-LR on a complete undirected graph $G = (N, E)$, where $N = \{0\} \cup N'$ contains the supplier (depot) 0 and the customer set $N' = \{1, \ldots, n\}$. Each edge $\langle i, j \rangle \in E$ has a non-negative routing cost c_{ij}. The planning horizon is $T = \{1, \ldots, |T|\}$, and the homogeneous vehicle fleet is $K = \{k_1, \ldots, k_{|K|}\}$, with each vehicle having capacity Q.

Each customer $i \in N'$ has initial inventory I_{i0}, maximum inventory capacity U_i, and demand r_{it} in period t. The supplier has initial inventory I_{00} and makes r_{0t} units available in period t. We assume that supplier availability is sufficient to meet customer demand over the planning horizon, and that customer demand is deterministic and known in advance.

For each customer i, vehicle k, and period t, q_{it}^k denotes the quantity delivered and z_{it}^k indicates whether vehicle k visits customer i. The variable y_{ij}^{kt} denotes the number of times vehicle k traverses edge $\langle i, j \rangle$ in period t. Split deliveries are not permitted: each customer can be served by at most one vehicle

in a period. For $S \subseteq N'$, let

$$E(S) = \{\langle i,j\rangle \in E : i,j \in S\}.$$

We set $\mu_{i0} = 0$ for each customer. Assuming that the total delivered quantity is positive, the IRP-LR is formulated as follows:

$$\min \quad \frac{\displaystyle\sum_{k\in K}\sum_{\langle i,j\rangle\in E}\sum_{t\in T} c_{ij} y_{ij}^{kt}}{\displaystyle\sum_{k\in K}\sum_{i\in N'}\sum_{t\in T} q_{it}^{k}} \tag{2}$$

$$\text{s.t.} \quad I_{0t} = I_{0,t-1} + r_{0t} - \sum_{k\in K}\sum_{i\in N'} q_{it}^{k}, \qquad t \in T, \tag{3}$$

$$I_{it} - \mu_{it} = I_{i,t-1} - \mu_{i,t-1} - r_{it} + \sum_{k\in K} q_{it}^{k}, \quad i \in N',\ t \in T, \tag{4}$$

$$I_{it} \geq 0, \qquad i \in N,\ t \in T, \tag{5}$$

$$\mu_{it} \geq 0, \qquad i \in N',\ t \in T, \tag{6}$$

$$\sum_{i\in N'}\sum_{t\in T} \mu_{it} \leq 0, \tag{7}$$

$$\sum_{k\in K} q_{it}^{k} \leq U_i - I_{i,t-1}, \qquad i \in N',\ t \in T, \tag{8}$$

$$q_{it}^{k} \leq U_i z_{it}^{k}, \qquad i \in N',\ k \in K,\ t \in T, \tag{9}$$

$$\sum_{i\in N'} q_{it}^{k} \leq Q z_{0t}^{k}, \qquad k \in K,\ t \in T, \tag{10}$$

$$\sum_{k\in K} z_{it}^{k} \leq 1, \qquad i \in N',\ t \in T, \tag{11}$$

$$\sum_{j:\langle i,j\rangle\in E} y_{ij}^{kt} = 2 z_{it}^{k}, \qquad i \in N,\ k \in K,\ t \in T, \tag{12}$$

$$\sum_{\langle i,j\rangle\in E(S)} y_{ij}^{kt} \leq |S| - 1, \qquad S \subseteq N',\ |S| \geq 2,\ k \in K,\ t \in T, \tag{13}$$

$$z_{it}^{k} \in \{0,1\}, \qquad i \in N,\ k \in K,\ t \in T, \tag{14}$$

$$q_{it}^{k} \geq 0, \qquad i \in N',\ k \in K,\ t \in T, \tag{15}$$

$$y_{ij}^{kt} \in \{0,1,2\}, \qquad \langle i,j\rangle \in E,\ k \in K,\ t \in T. \tag{16}$$

The objective (2) minimizes total routing cost per total quantity delivered. Constraints (3) and (4) impose inventory balance at the supplier and customers, respectively. Since $\mu_{it} \geq 0$, constraint (7) forces $\mu_{it} = 0$ for every customer and period in the original problem. Consequently, (4) reduces to the standard customer inventory balance and (5) prevents stockouts. Constraint (8) limits deliveries according to customer inventory capacity. Constraints (9)–(11) link deliveries to visits, impose vehicle capacity, and prohibit split deliveries. Constraints (12)–(13) define feasible vehicle routes. Since (13) contains exponentially many inequalities, these constraints may be generated dynamically.

4 Background and Literature

Table 1 distinguishes studies of the canonical single-supplier IRP-LR from related ratio-based variants, the richer ROADEF/EURO challenge, and conventional IRPs whose methods provide algorithmic context. This distinction matters because studies of related problems can inform algorithm design without being direct comparisons for the formulation in Section 3.

The exact methods of Archetti et al. (2017, 2019) establish the fractional-optimization basis of the canonical IRP-LR. The heuristic study of Alvarez et al. (2018) is the direct metaheuristic precedent: it already applies iterated local search and simulated annealing to the logistic-ratio objective. The methods developed for the ROADEF/EURO challenge address a richer industrial problem, while the studies of Alonso-Pecina et al. (2024); Man et al. (2025) consider related IRP variants with different problem structures or ratio definitions.

Recent algorithms for conventional cost-based IRPs provide further methodological context. For example, Castro et al. (2026) combine hybrid genetic search with a network-flow procedure for inventory decisions, while Zhao et al. (2025) develop a delivery-schedule neighborhood and consider penalized unmet demand. These approaches demonstrate the value of integrating route search with specialized inventory or delivery-schedule procedures. However, a procedure designed for an absolute-cost objective does not automatically optimize the LR: delivery quantities also affect the denominator of (2). Likewise, the general principle of exploring infeasible solutions is established in IRP research; its implementation and integration for the IRP-LR are methodological choices to be evaluated in the present study.

4.1 Research Gap

The literature does not leave a gap for the first heuristic or the first iterated local search for the IRP-LR: Alvarez et al. (2018) already proposed both ILS and simulated annealing for a logistic-ratio objective. Instead, the relevant gap is the limited development and evaluation of heuristics specifically designed for the interaction between routing decisions and the decision-dependent denominator of the IRP-LR objective. In particular, the methods developed for conventional cost-based IRPs cannot be transferred without considering how quantity changes affect the ratio. This paper addresses that algorithm-design gap through a constructive heuristic and an MSILS that combines quantity-focused disturbance, controlled exploration of inventory-infeasible solutions, and local rebalancing of vehicle-capacity violations.

4.2 Contributions

This paper develops and evaluates a multi-start iterated local search (MSILS) for the single-supplier IRP-LR. Its contributions are:

- We develop a constructive heuristic tailored to the cost-per-unit-delivered objective and to the coupled routing and replenishment decisions of the

Table 1: Selected literature relevant to the IRP-LR and the proposed solution methodology.

Study	**Problem / objective**	**Solution approach**	**Key search characteristic**	**Relevance**
Archetti et al. (2012); Archetti and Speranza (2016); Bertazzi and Speranza (2012); Coelho et al. (2014)	Classical IRP; absolute cost	IRP formulations, methods, and review	Integrates routing and inventory decisions	Background for the classical IRP
Archetti et al. (2017)	Single-supplier IRP-LR	Exact fractional optimisation	Joint routing and replenishment optimisation	Foundational academic IRP-LR formulation
Alvarez et al. (2018)	IRP with LR objective	ILS and simulated annealing	Neighbourhood search for routing and replenishment	Direct heuristic precedent and principal baseline
Archetti et al. (2019)	Single-supplier IRP-LR	Iterative exact method	Solves IRPs with modified linear objectives	Exact-method benchmark for the IRP-LR
Absi et al. (2020); Su et al. (2020); He et al. (2020); Kheiri (2020); André et al. (2020)	Rich ROADEF/EURO IRP	Branch-cut-and-price, matheuristics, and hyper-heuristic	Integrates routing, delivery, and scheduling decisions	Methods for the industrial challenge variant
Baldacci et al. (2020); Dinkelbach (1967); Prangprakhon and Nimana (2026)	Fractional routing or general fractional optimisation	Exact, parametric, and subgradient methods	Develops methods for fractional objectives	General mathematical context; not IRP-specific
Alonso-Pecina et al. (2024); Man et al. (2025)	Related ratio-based IRP variants	ILS and robust fractional optimisation	Multiple sources, multiple products, uncertainty, or transshipment	Related LR applications with different assumptions or ratio definitions
Laganà et al. (2024); Castro et al. (2026); Zhao et al. (2025)	Conventional cost-based IRPs	ILS matheuristic and hybrid genetic search	Combines route search with inventory or delivery-schedule optimisation	Recent methodological context; different objective
Chiu et al. (2024); Dinh et al. (2025); Iswari et al. (2026); Ekşi et al. (2026)	IRP variants and general IRP literature	Linear relaxation, matheuristics, integrated modelling, and review	Addresses long-term cost, delivery policies, urban decisions, and IRP methods	Contextual work; not direct IRP-LR comparisons

IRP-LR. This is an algorithmic contribution beyond the existing heuristic precedent of Alvarez et al. (2018).

- We develop an MSILS that combines iterated local search with quantity-focused disturbance and an adaptive penalty mechanism that allows controlled exploration of solutions with customer stockouts.
- We introduce a local rebalance mechanism for vehicle-capacity violations and integrate it into the search. The method draws on the broader feasible/infeasible search principle used in hybrid genetic search of Vidal (2022), while its contribution here is its adaptation to the representation and objective of the IRP-LR.
- We provide an expanded computational evaluation against the direct IRP-LR heuristic baseline of Alvarez et al. (2018). The results show that the proposed method improves solution quality on almost all tested instances, with larger improvements on the larger instances. Where available, we also compare the heuristic solutions with published optimal values.

5 Description of the Algorithm

This section describes the proposed Multi-Start Iterated Local Search (MSILS) algorithm for solving the Inventory Routing Problem with Logistic Ratio (IRP-LR). The algorithm uses features that include a mechanism for delivery quantity adjustment, delivery quantity-focused local search operators, and exploring infeasible regions through the amalgamation of iterated local search and Lagrangian relaxation. The remainder of this section is organised as follows. Section 5.1 presents the overall MSILS framework. Section 5.2 describes the constructive heuristic used to generate the initial solutions. Section 5.3 provides the details on the local search procedure for solving the relaxed problem. Section 5.4 introduces a quantity-focused rebalancing procedure that attempts to improve total delivery quantity while preserving route schedules. Finally, Section 5.5 describes the perturbation mechanism used to diversify the search and escape local optima.

5.1 Main algorithm

Let s denote a solution, which may be infeasible for the original problem (2)-(16). For each customer $i \in N'$, each time period $t \in T$, and each vehicle $k \in K$, part of the solution s is characterised by $q_{it}^k(s)$, $I_{it}(s)$, and $\mu_{it}(s)$, where

- $q_{it}^k(s) \geq 0$ is the delivery quantity by vehicle k, to customer i in time period t;
- $I_{0t}(s) \geq 0$ is the inventory level at the depot in time period t;
- $I_{it}(s) \geq 0$ is the inventory level at customer i in time period t; and

- $\mu_{it}(s) \geq 0$ is the stockout amount at customer i in time period t.

If customer i is visited by vehicle k in time t, $q_{it}^k(s)$ is positive; zero otherwise. The net inventory level is defined as $I_{it}(s) - \mu_{it}(s)$ and is computed recursively according to

$$I_{i,1}(s) - \mu_{i,1}(s) = I_{i,0} - r_{i,1} + \sum_{k \in K} q_{i1}^k(s), \tag{17}$$

$$I_{i,t}(s) - \mu_{i,t}(s) = I_{i,t-1}(s) - \mu_{i,t-1}(s) - r_{i,t} + \sum_{k \in K} q_{it}^k(s),\ 2 \leq t \leq |T|. \tag{18}$$

The inventory level of the depot is computed by

$$I_{0,1}(s) = I_{0,0} + r_{0,1} - \sum_{i \in N'} \sum_{k \in K} q_{i1}^k(s), \tag{19}$$

$$I_{0,t}(s) = I_{0,t-1}(s) + r_{0,t} - \sum_{i \in N'} \sum_{k \in K} q_{it}^k(s),\ 2 \leq t \leq |T|. \tag{20}$$

If s is feasible to the relaxed problem and has $\sum_{i \in N'} \sum_{i \in T} \mu_{it}(s) = 0$, then s is also feasible for the original problem. The value of s on the objective function (2) is denoted by $f(s)$, and the value on the objective function (24) with Lagrange multiplier α is denoted by $f_\mu(s, \alpha)$. The following pseudocode outlines the proposed MSILS.

Algorithm 1 MSILS

$s^* \leftarrow \emptyset$, $f(s^*) \leftarrow +\infty$
for $i \leftarrow 1$ **to** $Max_{restart}$ **do**
  $s \leftarrow$Initial_Solution_Procedure, $\hat{s} \leftarrow s$, $it \leftarrow 0$
  if $f(s) < f(s^*)$ **then**
    $s^* \leftarrow s$
  end if
  while $it < |N'| + \sigma|T| + \phi|K|$ **do**
    $\alpha \leftarrow \epsilon(s^*, s, 0, \eta)$, $\bar{s} \leftarrow s$, $s \leftarrow$Local_Search(s, α)
    while $f_\mu(s, \alpha) \neq f_\mu(\bar{s}, \alpha)$ **or** s has stockout **do**
      $\alpha \leftarrow \epsilon(s^*, s, \alpha, \eta)$, $\bar{s} \leftarrow s$, $s \leftarrow$Local_Search(s, α)
    end while
    if s is feasible **and** $f(s) < f(\hat{s})$ **then**
      $\hat{s} \leftarrow s$, $it \leftarrow 0$, $\hat{s} \leftarrow$Rebalance$(\hat{s})$
    end if
    if $\hat{s}$ is feasible **and** $f(\hat{s}) < f(s^*)$ **then**
      $s^* \leftarrow \hat{s}$
    end if
    $s \leftarrow$ Disturbance$(\hat{s}, it)$, $it \leftarrow it + 1$
  end while
end for
return s^*

Initially, the global best-known solution s^* is empty, and $f(s^*)$ is set to $+\infty$ (line 1). s^* is updated throughout the algorithm. The FOR loop (lines 2-20) implements the multi-start framework. At each restart, a feasible solution to the original problem is generated using the constructive heuristic described in 5.2. This solution is used as the starting solution for the upcoming application of the iterated local search (WHILE loop, lines 7-19). By repeatedly restarting the iterated local search from different initial solutions, the algorithm can explore diversified regions of the solution space. The number of starting solutions is controlled by the parameter $Max_{restart}$.

Each application of the iterated local search uses a local search procedure to solve a sequence of relaxed problems, each with a different value of α. If an optimal solution $\tilde{s}$ for the relaxed problem can be obtained, then following a commonly used version of the Lagrangian relaxation method Fisher (1981), α is computed by

$$\alpha + \frac{\eta \times (f(s^*) - f_\mu(\tilde{s}, \alpha)) \times (\sum_{i \in N'} \sum_{t \in T} \mu_{it}(\tilde{s}))}{\left(\sum_{i \in N'} \sum_{t \in T} \mu_{it}(\tilde{s})\right)^2} \tag{21}$$

where η is a scalar. Since the local search cannot guarantee the optimal solution $\tilde{s}$, instead of (21), α is computed by

$$\epsilon(s^*, s, \alpha, \eta) = \alpha + \frac{\eta \times \delta \times (\sum_{i \in N'} \sum_{t \in T} \mu_{it}(s) + 0.001)}{\left(\sum_{i \in N'} \sum_{t \in T} \mu_{it}(s)\right)^2 + 0.00001} \tag{22}$$

where

$$\delta = \begin{cases} f(s^*) - f_\mu(s, \alpha) & \text{if } f(s^*) - f_\mu(s, \alpha) > 0; \\ 0.1 \times f(s^*) & \text{Otherwise.} \end{cases}$$

Once local search obtains a feasible solution for the original problem, $\epsilon(\cdot)$ becomes just

$$\alpha + \frac{\eta \times \delta \times (0.001)}{0.00001}$$

which greedily increases α until the stopping criterion is satisfied for the WHILE loop (lines 9-11). The initial value of α is computed by $\epsilon(s^*, s, 0, \eta)$ in line 8 and the subsequent values is computed by $\epsilon(s^*, s, \alpha, \eta)$ in line 10 using α from the previous call of the local search procedure.

To escape from the local optimum, the MSILS begins with a quantity-focused disturbance procedure, referred to as the rebalance procedure. This procedure attempts to further improve the locally best-known solution $\hat{s}$ by adjusting the delivery quantities, keeping the schedule fixed (see Section 5.4). The output of the procedure is either the input, meaning that the procedure failed to make any improvement or a feasible solution of the original problem with a lower logistic ratio. To limit computational overhead, the rebalance procedure is invoked only when $\hat{s}$ has been improved. To further diversify the search, the MSILS applies a route-focused disturbance procedure on $\hat{s}$, generating the starting solution for the subsequent call of local search. The iterated local search (WHILE loop lines 7-19) terminates after the counter it reaches $|N'| + \sigma|T| + \phi|K|$, the maximum

permissible number of consecutive iterations without improvement, where σ and ϕ are the weight parameters on the number of time periods and the number of vehicles, respectively.

5.2 Initial feasible solution

This section describes the heuristic to construct the initial solution for the MSILS. It aims to produce a good feasible solution to the original problem (2)-(16), thereby establishing the initial upper bound.

The solution s is initialised with empty routes and zero delivery quantities, $q_{it}^k(s) = 0$ for $i \in N'$, $k \in K$, and $t \in T$. The net inventory levels $I_{it}(s) - \mu_{it}(s)$ are calculated by (19) and (20) accordingly. The customers are grouped by the earliest time period in which their net inventory levels become negative. Those customers with earlier stockouts are given higher priority. Within each stockout group, customers are further partitioned spatially. Let (x_i, y_i) denote the coordinates of location $i \in N$. Using the coordinate minima and maxima

$$x_{\min} = \min_{i \in N} x_i, \quad x_{\max} = \max_{i \in N} x_i,$$

$$y_{\min} = \min_{i \in N} y_i, \quad y_{\max} = \max_{i \in N} y_i,$$

an axis-aligned bounding rectangle is defined with corners (x_{min}, y_{min}), (x_{max}, y_{min}), (x_{max}, y_{max}), and (x_{min}, y_{max}). The resulting axis-aligned rectangle is divided into a $d \times d$ grid of subregions $R = \{r_1, r_2, \ldots, r_{d \times d}\}$ where d is randomly selected between the minimum possible grid resolution d_{min} and the maximum possible grid resolution $d_{\max}$.

Observe that, during the early stages of the solution construction, vehicles typically have sufficient capacity to deliver the largest possible quantities respecting both vehicle and inventory capacity constraints. Such flexibility reduces as the construction goes on, and customers who will be considered at a later stage often require more visits. Thus, the subregions are ordered by decreasing distance from their centres to the depot, so customers in more distant subregions are processed earlier.

For each stockout group and subregion, the heuristic randomly selects a customer and repeatedly schedules a visit to address that customer's earliest remaining stockout. It first searches for an unoccupied period between the current search period and the stockout period that has remaining fleet capacity, choosing randomly among available periods. If none is available, it searches backwards for an earlier period. In the selected period, it assigns the customer to the vehicle that can accommodate the largest delivery, with the quantity limited by that vehicle's residual capacity and the customer's remaining inventory capacity. It then updates the customer's inventory trajectory and continues until no customer stockout remains. If a customer faces stockout even when the customer is visited in every period, then no feasible schedule exists for that customer. In this case, the instance is infeasible. The procedure returns an empty solution, and the MSILS is terminated.

Let $P_{t,r} \subseteq N'$ denote the subset of customers in priority group $t \in T$ and subregion $r \in R$. The pseudocode, Construct Initial Solution, summarises the procedure.

Construct Initial Solution

initialise solution s
partition customers by priority and distance to depot
for $t \in T$ **and** $r \in R$ **do**
  while $P_{t,r} \neq [\,]$ **do**
    randomly choose a customer i from $P_{t,r}$
    $t_{current} \leftarrow 1$
    while $\{\hat{t} \in T : I_{i,\hat{t}}(s) - \mu_{i,\hat{t}}(s) < 0\} \neq [\,]$ **do**
      $t_{stockout} \leftarrow \min\{t \in T : I_{i,t}(s) - \mu_{i,t}(s) < 0\}$
      $t_{assign} \leftarrow$ Find Feasible Visit Time $(t_{current}, t_{stockout}, i)$
      $(k, q) \leftarrow$ Find Vehicle To Deliver Max(t_{assign}, i, s)
      $q^k_{i,t_{assign}} \leftarrow q$
      add customer i to vehicle k's route on time period t_{assign}
      for all $t' \in T$, update $I_{i,t'}(s) - \mu_{it'}(s)$ according to (19) and (20)
      $t_{current} \leftarrow t_{assign} + 1$
    end while
    $P_{t,r} \leftarrow P_{t,r} \setminus i$
  end while
end for
$s \leftarrow$nearest_neighbour_insertion(s)
return s

After the schedule has been constructed, a nearest-neighbour insertion Bräysy and Gendreau (2005) is applied independently to each vehicle's route in each period. It starts with the customer nearest to the depot. At each subsequent step, it selects the unvisited customer closest to the current tour and inserts that customer in the position that yields the smallest increase in the length of the closed route. The resulting solution is then returned to MSILS.

5.3 Local search

The local search uses five neighbourhood operators, denoted by O_1, O_2, O_3, O_4, and O_5. These operators are based on those commonly found in the field of vehicle routing (see, for example, Kindervater and Savelsbergh (1997), Laporte et al. (2000), and Elshaer and Awad (2020)) and are also frequently used in the field of inventory routing (see, for example, Alvarez et al. (2018), Laganà et al. (2024), and Castro et al. (2026)). Each operator defines a neighbourhood through a set of alterations. Each alteration transforms the input solution into a neighbouring solution by modifying one or more of its components.

Rather than employing separate operators for route alterations and delivery quantity alterations, the operators O_1, O_2, O_3, and O_4 integrate both into a

single alteration. Such an integration enables the exploration of the logistic ratio, i.e., the simultaneous exploration of total transportation cost and total delivery quantity. The routing component of these operators is described as follows:

- Operator O_1 moves up to 3 adjacent customers from a route to another in the same period.
- Operator O_2 exchanges up to 2 adjacent customers in a route with up to 2 adjacent customers in another route in the same period.
- Operator O_3 transfers a customer visit to a different time period in which this customer is not visited, with the vehicle giving the largest possible delivery quantity at a position with the lowest in transportation cost.
- Operator O_4 exchanges a customer in a route with one unvisited customer, including the case where either a visited customer becomes unvisited or an unvisited customer becomes visited.

For an alteration that results in removing the visit to customer $i \in N'$ at time $t \in T$ on vehicle $k \in K$, the associated $q_{it}^k(s)$ becomes 0. For an alteration that results in a new visit to customer $i \in N'$ at time $t \in T$ on vehicle $k \in K$, the associated $q_{it}^k(s)$ is computed by

$$q_{jt}^k = \max\{0, \min\{Q - \sum_{i \in N'} q_{it}^k(s), U_i - I\}\} \tag{23}$$

where

$$I = \begin{cases} I_{i,0} & \text{if } t = 1; \\ I_{i,t-1}(s) & \text{if } 2 \leq t \leq |T| \end{cases}$$

Please note that $\max\{0, \cdot\}$ is required since the local search allows the violation of stockout where I can be negative and $U_i - I$ can also be negative. For customer i, the resulting inventory level is updated recursively according to (19) and (20). These updated inventory levels may affect the subsequent deliveries. Therefore, following the update in each time period, if the customer is visited in the subsequent time period, the delivery quantity for that visit is adjusted using (23).

For the last operator, Operator O_5 is a pure routing operator. It transfers up to 3 adjacent customers from their current position to another position in the same route. In contrast to operators O_1, O_2, O_3, and O_4, the alterations associated with operator O_5 do not affect the delivery quantities and the inventory levels.

Lagrangian Relaxation for Infeasibility Exploration Constraint (7), together with $\mu_{it} \geq 0$, forces $\mu_{it} = 0$ for every customer and period in the original problem. We relax this constraint to allow the search to explore solutions with

Local Search(s, α)

for $i \leftarrow 0;\ i \leq 5;\ i \leftarrow i+1$ **do**
  repeat
    $\hat{s} \leftarrow s$, $j \leftarrow 1$
    while $j < |O_i(s)|$ **do**
      $s' \leftarrow s_j \in O^i(s)$, $j \leftarrow j+1$
      if $f_\mu(s', \alpha) < f_\mu(s, \alpha)$ **then**
        $s \leftarrow s'$
      end if
    end while
  until $f_\mu(s, \alpha) = f_\mu(\hat{s}, \alpha)$
end for
$s \leftarrow$ remove any visit with zero delivery quantity
return s

temporary inventory shortages. Dualizing (7) with a multiplier $\alpha \geq 0$ gives the Lagrangian objective

$$\min \quad \frac{\sum_{k\in K}\sum_{\langle i,j\rangle\in E}\sum_{t\in T} c_{ij} y_{ij}^{kt}}{\sum_{k\in K}\sum_{i\in N'}\sum_{t\in T} q_{it}^{k}} + \alpha \sum_{i\in N'}\sum_{t\in T} \mu_{it}. \tag{24}$$

The relaxed problem consists of objective (24) subject to

$$(3) - -(6), \qquad (8) - -(16),$$

with constraint (7) removed.

Under customer balance (4), μ_{it} represents accumulated unmet demand at customer i by the end of period t. Therefore, $\sum_{i,t} \mu_{it}$ penalizes shortage carried across periods, rather than only the shortage occurring in each individual period. The parameter α controls the strength of this penalty: smaller values allow the search to consider solutions with larger shortages, while larger values place greater emphasis on returning to the stockout-free region.

The proposed MSILS adjusts α during the search and uses (24) to guide exploration of feasible and infeasible solutions. We use this Lagrangian objective as a search criterion; since the relaxed problems are not solved to global optimality at each search step, their heuristic values are not used as Lagrangian lower bounds. The original problem refers to formulation (2)–(16), and the relaxed problem refers to (24) with constraint (7) removed.

The local search procedure implements a sequential variable neighbourhood descent procedure for solving the relaxed problem. For a given solution s and the Lagrange multiplier α, the neighbourhoods induced by operators O_1 to O_5 are explored successively. Within a neighbourhood associated with Operator $i \in \{1, 2, 3, 4, 5\}$, all neighbouring solutions denoted by $O_i(s)$ are evaluated

according to the objective function $f_\mu(\cdot, \alpha)$, and the current solution is updated whenever an improving neighbour is identified. The neighbourhood is repeatedly re-explored until no improving solution can be found, thereby ensuring local optimality with respect to the corresponding operator. The search then proceeds to the next operator. The procedure terminates once all five operators have been exhausted without improvement, at which point any visits associated with zero delivery quantity are removed and the resulting solution is returned. The pseudocode, Local Search summaries the local search procedure.

5.4 Rebalance Procedure

The local search procedure simultaneously explores routing decisions and delivery quantities. However, the improved solution may still contain delivery quantities that are not fully consistent with the fixed schedule. In particular, some customer visits may carry quantities that are unnecessarily large, whereas other visits may have unused inventory or vehicle capacity that could be exploited to increase the total delivery quantity. Since the objective of the IRP-LR is expressed as the ratio between routing cost and delivery quantity, improving the quantity allocation alone can further improve the objective value even when the routing schedule remains unchanged. This section describes the rebalance procedure, which exploits such an opportunity whenever a new incumbent solution s^* is obtained.

The procedure consists of two phases. First, delivery quantities are partially removed from existing visits whenever such reductions do not create stockouts. This releases inventory and vehicle capacity that may have been inefficiently allocated. Second, the released quantities are reallocated among the existing visits. Customers are selected randomly, favouring customers with low inventory levels, high demand rates, long intervals until their next visits, and infrequent replenishment patterns. For each selected customer, the maximum feasible quantity is assigned while respecting inventory capacity and vehicle capacity. The inventory level is then updated, and visits associated with zero delivery quantity are subsequently removed.

By combining quantity reallocation with randomised customer selection, the rebalance procedure not only intensifies the search for better delivery quantities around a promising routing schedule but also introduces diversification, enabling the algorithm to escape local optima in the quantity domain. The pseudocode below summarises the rebalance procedure.

Algorithm 4 Rebalance(s^*)

```
s ← s*
for i ← 1 to |N'| do
    for t ← 1 to |T| do
        if customer i is visited on day t then
            Quantity ← Quantity - Feasible_Quantity_To_Removal(i, t)
            Update inventory levels for all subsequent days
        end if
    end for
end for
s ←Assign_max_possible_quantity_after_feasible(s)
s ←Remove visits with zero delivery quantities
return s
```

In addition to the rebalance procedure described above, an alternative Destory_And_Rebalance procedure was developed. This variant first removes all delivery quantities and then reconstructs them while keeping the routing schedule fixed. Because the reconstruction does not guarantee feasibility, a repair mechanism is subsequently applied to eliminate stockouts whenever possible. The larger perturbation induced by this procedure promotes diversification and allows the search to explore quantity allocations that are less accessible through the former rebalance procedure. The pseudocode for this variant is presented in Appendix D. Both rebalance procedures are examined in the computation experiments.

5.5 Perturbation

The perturbation procedure is a diversification mechanism designed to help the MSILS escape local optima. It randomly removes two or more adjacent customers from a route together with their associated delivery quantities, after which inventory levels and subsequent deliveries are updated accordingly. The number of removals is determined by $1 + \lfloor \frac{\theta \times it}{Max_{it}} \rfloor$, where it is the number of consecutive non-improving iterations, Max_{it} is the maximum number of consecutive non-improving iterations allowed, and θ is a parameter handling the disturbance strength. Hence, the perturbation strength increases progressively as it increases. This adaptive strategy allows the algorithm to balance intensification and diversification by exploring increasingly distant regions of the solution space when improvements become difficult to obtain. The resulting perturbed solution serves as the starting point for the next local search iteration. Since the removals typically induce stockouts, the perturbed solution is generally infeasible for the original problem. However, allowing such infeasibility enables the search to traverse infeasible regions of the solution space and escape deep local optima. Feasibility is subsequently restored during the local search.

6 Computational experiments

This section describes the computational experiments conducted to evaluate the performance of the proposed MSILS. The algorithm is implemented in C++, and all experiments were conducted on a computer with an Intel Core i7-1185G7 3.00 GHz CPU and 16GB RAM. The rest of the section is structured as follows. The test instances used for the computation experiments are summarised in Section 6.1. Section 6.3 discusses the parameter settings and demonstrates the performance gain from different components used in the MSILS. Section 6.4 evaluates the MSILS by comparing its performance with the performance of the ILS and SA algorithms proposed by Alvarez et al. (2018). This section also evaluates the performance gain from integrating the Hybrid Genetic Search (HGS) framework of Vidal (2022) into MSILS.

6.1 Test instances

To evaluate the proposed MSILS, computational experiments were conducted on benchmark instances originating from Archetti et al. (2007), Archetti et al. (2012), and the 12th DIMACS Implementation Challenge for the multi-vehicle IRP. These instances have been widely used in the IRP literature, including Archetti et al. (2012); Coelho and Laporte (2013); Desaulniers et al. (2016); Santos et al. (2016); Archetti et al. (2017); Alvarez et al. (2018); Archetti et al. (2019); Castro et al. (2026).

The benchmark introduced by Archetti et al. (2007) contains a set of single-vehicle IRP instances comprising 50 instances ranging from 5 to 50 customers for a planning horizon of 3 time periods, and 30 instances ranging from 5 to 30 customers for a planning horizon of 6 time periods. Each instance is available under two inventory-cost scenarios (high and low), resulting in a total of 160 instances. These single-vehicle IRP instances were extended to multi-vehicle versions with a number of vehicles ranging from 2 to 5, creating an additional 640 instances. Following Alvarez et al. (2018) and the DIMACS Implementation Challenge, the vehicle capacities for the multi-vehicle IRP instances were obtained by dividing the vehicle capacity in the corresponding single-vehicle instances by the number of vehicles and rounding down to the nearest lower integer.

The 12th DIMACS Implementation Challenge further introduced 160 additional multi-vehicle instances with a number of customers ranging from 35 to 50, completing the 6-time-period planning horizon dataset. In addition, the single- and multi-vehicle IRP instances for 3 planning horizons were extended to a planning horizon of 4 and 5 periods in Archetti et al. (2017). This was done by increasing the planning horizons from 3 to 4 and 5, resulting in another 1000 instances. Collectively, these datasets comprise 1960 instances covering 5 to 50 customers, 3 to 6 planning horizons, 1 to 5 vehicles, and two inventory-cost settings.

A larger scale of the benchmark was also considered. In Archetti et al. (2012), a set of 60 single-vehicle IRP instances was introduced with a number

of customers ranging from 50 to 200. These instances were also extended to multi-vehicle instances, resulting in a total of 300 instances. The single-vehicle IRP instances from Archetti et al. (2007) and Archetti et al. (2012) can be downloaded from Single-vehicle IRP instances and the instances for the 12th DIMACS Implementation Challenge for the multi-vehicle IRP can be downloaded from DIMACS Inventory Routing Challenge.

For the above-mentioned instances, the distance c_{ij} between each node $i \in N$ and $j \in N$ is calculated and rounded to the nearest integer by

$$\left\lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + \frac{1}{2} \right\rfloor$$

, where x_i, x_j, y_i, y_j are the x- and y-coordinates for nodes i and j. Since the IRP-LR does not include inventory cost, only one inventory-cost setting was retained, and inventory-cost information was discarded. Consequently, the final test bed consists of 980 instances for 5 to 50 customers and 150 instances for 50 to 200 customers.

Also noting, two instances were excluded from the study due to infeasibility. The instance "abs5n5_5" with 6 periods and 5 vehicles is a known infeasible instance Alvarez et al. (2018). In addition, the preliminary experiments revealed that the instance "abs4n5_5" with 5 planning periods and 5 vehicles is also infeasible, as stockouts occur even when a certain customer is visited in every period.

For reproducibility, all benchmark instances used in the computational experiments are publicly available at `https://github.com/AndyElhamSarang/IRP_instances.git`.

6.2 Parameter calibrations

The parameters for the MSILS are set to the values listed in Table 2. These values were determined from preliminary experiments.

Table 3: Variants derived from MSILS

Variants	Description
MSILS	Full algorithm described in Section 5.
$\not{\mathcal{R}}$	Deactivating the Rebalance Procedure at line 13 in MSILS.
$D\&R$	Instead of using the Rebalance Procedure described in Section 5.4, line 13 uses the Destory_And_Rebalance procedure described in Appendix D
$\not{\mathcal{D}}$	Deactivating the First-stage DISTURB from the procedure Perturbation.
MSILS-HGS	MSILS with HGS, with the time required for the HGS excluded from the time limit.
MSILS-Initial-Alt	Use constructive heuristics in Alvarez et al. (2018) to construct the initial feasible solution.

Table 2: Default values of parameters

Parameters	Definition	Tested Range
$d_{\min}$	Minimum grid resolution in initial solution construction	$\{1, \underline{\mathbf{2}}, 3, 4, 5\}$
$d_{\max}$	Maximum grid resolution in initial solution construction	$\{2, \underline{\mathbf{3}}, 4, 7\}$
$Max_{restart}$	Number of initial starting solutions	{1,**3**,5,10,20} {3,4,**5**,6,7,8,9}
η	Scalor for updating the Lagrange multiplier α	{ 1,3,**5**,10,20,100000}
σ	Weight parameters on number of time periods to compute the maximum permissible number of consecutive iterations without improvement from the ILS	{ 5,7,**10**,15,20}{**1**,2,3,4,5,6,8,10,12}
ϕ	Weight parameters on number of vehicles to compute the maximum permissible number of consecutive iterations without improvement from the ILS	{ 5,6,7,**8**,9,10,15,20}{8,**9**,10,11,12}
θ	Disturbance strength	{1,**2**,4,5,7,9,10}

6.3 Impact on the performance of individual mechanisms in MSILS and parameter tuning

The proposed MSILS has several key components, including an application of the Lagrangian relaxation technique, the rebalance procedure, and the disturbance procedure. This section examines the performance gain from utilising each of these components. Four variants have been derived from the MSILS. Each variant deactivates or modifies a specific component. Table 3 summarises each variant.

The variant $\not{\mathcal{L}}$ is used to examine the performance gain of using the Lagrangian relaxation technique. By initialising α to a large value, the Local_Search procedure would just repair the infeasible starting solution. Thereafter, due to the high penalty for stockout, the search process would remain in the feasible

solution space. All four variants and the MSILS were tested on Archettie2012 benchmark. Each version of the algorithm was given a time limit of 1 minute.

6.4 Performance evaluation

Compared with the supplementary results from Alvarez et al. (2018) available at https://www.dep.ufscar.br/munari/irp/.

The proposed MSILS is compared with ILS and SA in Alvarez et al. (2018). The algorithm is validated in small instances by comparing the optimal solutions obtained by the ACS algorithm developed in Archetti et al. (2019).

Table 4: Average percentage gap (%) of MSILS compared with ILS

	$\|T\| = 2$	$\|T\| = 3$	$\|T\| = 4$	$\|T\| = 5$	Average
$\|N'\| = 50$	-8.00	-4.30	-5.03	-5.13	-5.61
$\|N'\| = 100$	-8.63	-6.59	-6.57	-6.90	-7.17
$\|N'\| = 200$	-9.02	-6.16	-5.16	-4.10	-6.11
Average	-8.55	-5.68	-5.58	-5.38	-6.30

Table 5: Average percentage gap (%) of MSILS compared with SA

	$\|T\| = 2$	$\|T\| = 3$	$\|T\| = 4$	$\|T\| = 5$	Average
$\|N'\| = 50$	-7.59	-5.28	-4.82	-4.97	-5.67
$\|N'\| = 100$	-10.00	-6.66	-7.56	-6.16	-7.59
$\|N'\| = 200$	-15.59	-10.18	-9.77	-7.40	-10.73
Average	-11.06	-7.37	-7.38	-6.18	-8.00

6.5 Robustness

A Functions used to construct initial solution

Find Feasible Visit Time($t_{current}$, $t_{Stockout}$, i)

while true **do**
  if $t_{current} \leq t_{Stockout}$ **then**
    // **schedule forward**//
    $T_{candidate} \leftarrow \{t \in T : t_{current} \leq t \leq t_{stockout},$
      customer i is not visited at t,
      $Q \times |K| - \sum_{k \in K} \sum_{i \in N'} q_{i,t}^{k}(s) > 0\}$
    if $T_{candidate} \neq [\,]$ **then**
      return $t \sim \text{RANDOM}(T_{candidate})$
    else
      $t_{current} \leftarrow t_{Stockout} + 1$
    end if
  else
    // **schedule backward**//
    for $t \leftarrow t_{stockout}; t \geq 1; t \leftarrow t - 1$ **do**
      if customer i is not visited at time period t **then**
        if $Q \times |K| - \sum_{k \in K} \sum_{i \in N'} q_{i,t}^{k}(s) > 0$ **then**
          return t
        end if
      end if
    end for
  end if
end while

Find Vehicle To Deliver Max(t, i, s)

$k' \leftarrow |K| + 1$, $q' \leftarrow 0$,
for $k \in K$ **do**
  $q \leftarrow 0$
  if $t = 1$ **then**
    $q \leftarrow min\{Q - \sum_{j \in N'} q_{j,1}^{k}(s), U_i - I_{i,0}\}$
  else
    $q \leftarrow min\{Q - \sum_{j \in N'} q_{j,t}^{k}(s), U_i - I_{i,t-1}(s)\}$
  end if
  if $q' < q$ **then**
    $q' \leftarrow q$, $k' \leftarrow k$
  end if
end for
return k', q'

B MSILS with HGS

Algorithm 7 MSILS with HGS in Vidal (2022)

$s^* \leftarrow$ Empty **and** $f(s^*) \leftarrow +\infty$
for $i \leftarrow 1$ **to** $Max_{restart}$ **do**
  $s \leftarrow$ Construct initial solution
  $s \leftarrow HGS(s)$ {to improve routing}
  if $f(s) < f(s^*)$ **then**
    $s^* \leftarrow s$
  end if
  Run_HGS_Or_Not $\leftarrow$ FALSE
  $it \leftarrow 0$, $\eta \leftarrow \eta'$, $u \leftarrow 0$
  while $it < Max_{it}$ **do**
    $\alpha \leftarrow \epsilon(s^*, s, 0, \eta)$
    $\bar{s} \leftarrow s$
    $s \leftarrow$ Local_Search(s, α)
    while $f_\mu(s, \alpha) \neq f_\mu(\bar{s}, \alpha)$ **do**
      $\alpha \leftarrow \epsilon(s^*, s, \alpha, \eta)$
      $\bar{s} \leftarrow s$
      $s \leftarrow$Local_Search(s, α)
    end while
    if $min\{it, u\} \geq U$ **then**
      $u \leftarrow 0$, $\eta \leftarrow \kappa \times \eta$
    end if
    if s is feasible **and** $f(s) < f(s^*)$ **then**
      $s^* \leftarrow s$, $it \leftarrow 0$
      $s^* \leftarrow$ Rebalance(s^*)
      Run_HGS_Or_Not $\leftarrow$ TRUE
    end if
    $s \leftarrow$ Disturbance(s^*, it), $it \leftarrow it + 1$, $u \leftarrow u + 1$
  end while
  if Run_HGS_Or_Not = TRUE **then**
    $s^* \leftarrow HGS(s^*)$
  end if
end for
return solution s^*

C Functions used in Rebalance procedure guarantee feasible solution

Algorithm 8 Feasible_Quantity_To_Removal(i, j)

Quantity_To_Remove $\leftarrow min\{I_{i,j}, q_{i,j}\}$
$I \leftarrow I_{i,j-1} + q_{i,j} - r_{i,j}$−Quantity_To_Remove {New inventory level of period j}
Stockout $\leftarrow 0$
for $time \leftarrow j$ **to** $|T|$ **do**
  $I \leftarrow I_{i,time} + q_{i,j} - r_{i,j}$
  if Stockout $\leq -I'$ **then**
    Stockout $\leftarrow -I'$
  end if
end for
return $\leftarrow max\{0, Quantity - Stockout\}$

Algorithm 9 Assign_max_possible_quantity_after_feasible(s)

for $i \leftarrow 1$ **to** $|T|$ **do**
  PRIORITY $\leftarrow \emptyset$
  for $j \leftarrow 1$ **to** $|N'|$ **do**
    if customer j is visited on day i **then**
      Add customer j to PRIORITY with computed priority value
    end if
  end for
  while PRIORITY is not empty **do**
    Select a customer randomly according to PRIORITY weights
    Deliver the maximum feasible quantity to the selected customer
    Remove the selected customer from PRIORITY
    Update inventory levels and delivery quantity for subsequent days
  end while
end for
return s

D Rebalance procedure without guarantee feasible solution

Algorithm 10 Destory_And_Rebalance(s^*)

$s \leftarrow s^*$
Set all delivery quantities in s to zero
Update inventory levels in s
for $i \leftarrow 1$ **to** $|T|$ **do**
  for $j \leftarrow 1$ **to** $|N'|$ **do**
    if customer j is visited on day i **then**
      Quantity $\leftarrow$ Find_max_quantity_avoid_stockout(j, i)
      Assign Quantity
      Update inventory levels for all subsequent days
    end if
  end for
end for
Stockout_resolved $\leftarrow$ FALSE
$\{s, Stockout_resolved\} \leftarrow$Repair$(s, Stockout_resolved)$
if Stock_resolved = FALSE **then**
  return s
end if
$s \leftarrow$Assign_max_possible_quantity_after_feasible(s)
Remove visits with zero delivery quantities
return s

Algorithm 11 Find_max_quantity_avoid_stockout($customer, time$)

Quantity $\leftarrow 0$
Quantity $\leftarrow$ Find_max_stockout$(customer, time)$
Quantity $\leftarrow \min\{Quantity, U_{customer} - \max\{I_{customer,time-1}, 0\}\}$ {If stockout occurs in the previous period, assumes the inventory level is 0.}
Quantity $\leftarrow \min\{Quantity, Q - Vehicle_load\}$
return Quantity

Algorithm 12 Find_max_stockout$(customer, time)$

```
1: Largest_stockout ← 0
2: for t ← time to |T| do
3:    if Largest_stokout > I_{customer,t} then
4:       Largest_stockout ← I_{customer,t}
5:    end if
6: end for
7: Largest_stockout ← −Largest_stockout
8: return Largest_stockout
```

Algorithm 13 Repair$(s, Stockout_resolved)$

```
1: Possible_repair ← TRUE
2: while Stockout_resolved = FALSE and Possible_repair = TRUE do
3:    Stockout_resolved ← TRUE
4:    Possible_repair ← FALSE
5:    for i ← 1 to |T| do
6:       for j ← 1 to |N'| do
7:          if customer j has stockout at time i then
8:             Stockout_resolved ← FALSE
9:             Quantity_stockout ← I_{j,i}
10:            for k ← 1 to time i do
11:               if Customer j is visited in time k then
12:                  for customer z ∈ N' are visited by the same vehicle do
13:                     if Some quantity, Δ associated with customer z can be removed then
14:                        for x ← 1 to |T| do
15:                           if customer z is visited on time x then
16:                              if Δ can be delivered on time x to customer z without stockout then
17:                                 Possible_repair ← TRUE
18:                                 Quantity_Δ ← min{Quantity_stockout, Δ}
19:                                 Remove Quantity_Δ from visiting customer z on time k
20:                                 Deliver extra Quantity_Δ unit on day x
21:                                 Deliver Quantity_Δ unit for customer j on time k
22:                                 Update delivery quantity and inventory level
23:                              end if
24:                           end if
25:                        end for
26:                     end if
27:                  end for
28:               end if
29:            end for
30:         end if
31:      end for
32:   end for
33: end while
34: return {s, Stockout_resolved}
```

References

Absi, N., Cattaruzza, D., Feillet, D., Ogier, M., and Semet, F. (2020). A heuristic branch-cut-and-price algorithm for the ROADEF/EURO challenge on inventory routing. *Transportation Science*, 54:313–329.

Alonso-Pecina, F., Hernández-Báez, I. Y., López-Díaz, R. E., and Cruz-Rosales, M. (2024). Iterated local search approach to a single-product, multiple-source, inventory-routing problem. *Mathematics*, 12(7):991.

Alvarez, A., Munari, P., and Morabito, R. (2018). Iterated local search and simulated annealing algorithms for the inventory routing problem. *International Transactions in Operational Research*, 25(6):1785–1809.

André, J., Bourreau, E., and Wölfler Calvo, R. (2020). Introduction to the special section: ROADEF/EURO challenge 2016—inventory routing problem. *Transportation Science*, 54:299–301.

Archetti, C., Bertazzi, L., Hertz, A., and Speranza, M. G. (2012). A hybrid heuristic for an inventory routing problem. *INFORMS Journal on Computing*, 24(1):101–116.

Archetti, C., Bertazzi, L., Laporte, G., and Speranza, M. G. (2007). A branch-and-cut algorithm for a vendor-managed inventory-routing problem. *Transportation Science*, 41(3):382–391.

Archetti, C., Coelho, L. C., and Speranza, M. G. (2019). An exact algorithm for the inventory routing problem with logistic ratio. *Transportation Research Part E: Logistics and Transportation Review*.

Archetti, C., Desaulniers, G., and Speranza, M. G. (2017). Minimizing the logistic ratio in the inventory routing problem. *EURO Journal on Transportation and Logistics*, 6(4):289–306.

Archetti, C. and Speranza, M. G. (2016). The inventory routing problem: the value of integration. *International Transactions in Operational Research*, 23(3):393–407.

Baldacci, R., Lim, A., Traversi, E., and Wölfler Calvo, R. (2020). Optimal solution of vehicle routing problems with fractional objective function. *Transportation Science*, 54:434–452.

Bertazzi, L. and Speranza, M. G. (2012). Inventory routing problems: An introduction. *EURO Journal on Transportation and Logistics*, 1:307–326.

Bräysy, O. and Gendreau, M. (2005). Vehicle routing problem with time windows, part i: Route construction and local search algorithms. *Transportation Science*, 39(1):104–118.

Castro, B., Poggi, M., and Martinelli, R. (2026). A hybrid genetic search for the inventory routing problem. *Computers & Operations Research*, page 107376.

Chiu, A., Angulo, G., and Larrain, H. (2024). Optimizing the long-term costs of an inventory routing problem using linear relaxation. *Transportation Research Part E: Logistics and Transportation Review*, 183:103447.

Coelho, L. C., Cordeau, J.-F., and Laporte, G. (2014). Thirty years of inventory routing. *Transportation science*, 48(1):1–19.

Coelho, L. C. and Laporte, G. (2013). A branch-and-cut algorithm for the multi-product multi-vehicle inventory-routing problem. *International Journal of Production Research*, 51(23-24):7156–7169.

Desaulniers, G., Rakke, J. G., and Coelho, L. C. (2016). A branch-price-and-cut algorithm for the inventory-routing problem. *Transportation Science*, 50(3):1060–1076.

Dinh, N. M., Archetti, C., and Bertazzi, L. (2025). Matheuristic algorithms for the inventory routing problem with unsplit and split deliveries. *Networks*, 86(1):57–70.

Dinkelbach, W. (1967). On nonlinear fractional programming. *Management Science*, 13(7):492–498.

Ekşi, A., Elyasi, M., Minner, S., Van Woensel, T., Özener, O. Ö., and Ekici, A. (2026). Inventory-routing problems: Review, challenges, and innovations. *European Journal of Operational Research*.

Elshaer, R. and Awad, H. (2020). A taxonomic review of metaheuristic algorithms for solving the vehicle routing problem and its variants. *Computers & Industrial Engineering*, 140:106242.

Fisher, M. L. (1981). The lagrangian relaxation method for solving integer programming problems. *Management Science*, 27(1):1–18.

He, Y., Artigues, C., Briand, C., Jozefowiez, N., and Ngueveu, S. U. (2020). A matheuristic with fixed-sequence reoptimization for a real-life inventory routing problem. *Transportation Science*, 54(2):355–374.

Iswari, T., Caris, A., and Braekers, K. (2026). Extending the inventory routing problem to support integrated decision-making in an urban distribution network. *Networks*, 87(3):289–311.

Kheiri, A. (2020). Heuristic sequence selection for inventory routing problem. *Transportation Science*, 54:302–312.

Kindervater, G. A. P. and Savelsbergh, M. W. P. (1997). Vehicle routing: handling edge exchanges. *Local Search in Combinatorial Optimization*, pages 337–360.

Laganà, D., Malaguti, E., Monaci, M., Musmanno, R., and Paronuzzi, P. (2024). An iterated local search matheuristic approach for the multi-vehicle inventory routing problem. *Computers & Operations Research*, 169:106717.

Laporte, G., Gendreau, M., Potvin, J. Y., and Semet, F. (2000). Classical and modern heuristics for the vehicle routing problem. *International Transactions in Operational Research*, 7(4-5):285–300.

Man, X., Chang, D., Peng, Z., and Han, W. (2025). A logistics ratio-optimised inventory routing problem with lateral transhipment using mixed-integer robust fractional programming. *International Journal of Production Research*, 63:10381–10412.

Prangprakhon, M. and Nimana, N. (2026). Subgradient splitting methods for nonsmooth fractional programming with fixed-point constraints. *Journal of Scientific Computing*, 108(2):45.

Santos, E., Ochi, L. S., Simonetti, L., and González, P. H. (2016). A hybrid heuristic based on iterated local search for multivehicle inventory routing problem. *Electronic Notes in Discrete Mathematics*, 52:197–204.

Su, Z., Lü, Z., Wang, Z., Qi, Y., and Benlic, U. (2020). A matheuristic algorithm for the inventory routing problem. *Transportation Science*, 54(2):330–354.

Vidal, T. (2022). Hybrid genetic search for the cvrp: Open-source implementation and swap* neighborhood. *Computers & Operations Research*, 140:105643.

Zhao, J., Archetti, C., Pham, T. A., and Vidal, T. (2025). Large neighborhood and hybrid genetic search for inventory routing problems. *European Journal of Operational Research*.